Ders 5 Çalışma Özeti

Ders 5 Çalışma Özeti

Genel Konular

- Slaytlar İngilizce; hoca slayt kısımlarını çoğunlukla İngilizce, açıklamaları Türkçe anlattı.
  - Asıl ders yaklaşık 00:04'te başlar. İki kez 10 dakikalık ara verildi (yaklaşık 01:08 ve 02:10).
- Önceki dersin tekrarı
  - Sınıf (soyut) ve nesne (somut) farkı. Alan (member, field, attribute, property) ve metot (method, function, action) terimleri.
  - UML sınıf ve nesne diyagramları. Erişim sembolleri: + public, - private, # protected, ~ package.
  - Düzeltme: UML sıralama diyagramında istemcinin `getPlate` çağrısı bir istektir (request). Gelen cevap response/reply'dır.
- Paketler ve `import`
  - `import paket.*;` yalnız o paketin sınıflarını alır. Alt paketleri almaz.
- Bilgi gizleme (information hiding) ve `static`, `final` tekrarı
- İlkel tipler (primitive) ve sarmalayıcı sınıflar (wrapper class)
  - `byte`, `short`, `int`, `long`, `float`, `double`, `char`, `boolean` ve sarmalayıcıları.
  - `Integer.parseInt`, `Integer.valueOf`, `compareTo`, `NumberFormatException`.
- `String` sınıfı ve metotları
  - `compareTo`, `equals`, `charAt`, `length`, `concat`, `contains`, `endsWith`, `indexOf`, `split`, `substring`, `toUpperCase`, `trim`.
  - String sabit havuzu (string constant pool).
- `Math` ve `Random` sınıfları
- Giriş/çıkış: `System.out`, `Scanner`, `nextInt` sonrası `nextLine` hatası
- Metot parametreleri: Java'da parametre aktarımı her zaman değerle aktarımdır (call by value). Nesnelerde referansın değeri kopyalanır.
- Diziyi metoda gönderme, `for-each` döngüsü
- `toString()` metodunu ezme (override)

Hocanın Özellikle Vurguladığı Kısımlar

- **UML çok önemli.** UML diyagramından kod yazmayı, koddan UML çizmeyi bilin.
  - UML'de önce erişim belirteci yazılır. Sonra ad yazılır.
  - Alan: `- ad: String`. Metot: `+ getAd(): String`. Parametre: `ad: tip`.
- **Bütün alanları `private` yapın.**
  - Değişmeyecek alanlar (şase numarası, ISBN) yalnız constructor ile atanır.
  - Değişebilir alanlar (fiyat gibi) set/get metotlarıyla değiştirilir.
- Hoca `import` ile her şeyi dahil etmemeyi önerdi ve bunu C'deki `#include` ön işlemine benzetti. Bu benzetme Java'da geçerli değildir; ders sonrası notlara bakın.
- `nextInt()` sonrası `nextLine()` okunamıyor. Sebep: Enter tuşunun ürettiği satır sonu tamponda (buffer) kalır.
- Kitaplardaki "metoddan önce/sonra yazdır" örnekleri çoğu zaman gereksizdir. Önemli olan şudur: metoda giren veri metoddan değişmiş olarak çıkıyor mu?
- GUI kısmı sınavda pek sorulmaz; projede kullanılır.

Kısa Tekrar Notları

- Sınıf soyut, nesne somuttur. UML'de nesne adının altı çizilir.
- `import java.util.*;` alt paketleri dahil etmez.
- Tam sayı boyutları: `byte` 1, `short` 2, `int` 4, `long` 8 bayt. Küçükten büyüğe atama serbesttir; tersi derleme hatasıdır.
- Her ilkel tipin sarmalayıcısı vardır: `Integer`, `Long`, `Short`, `Byte`, `Double`, `Float`, `Character`, `Boolean`.
- `Integer.parseInt("12")` -> 12. Sayıya çevrilemeyen metinde `NumberFormatException` fırlatır.
- `new Integer(5)` kullanımdan kaldırıldı (deprecated). Yerine `Integer.valueOf(5)` kullanılır.
- `a.compareTo(b)`: `a < b` ise negatif, eşitse `0`, büyükse pozitif.
- `"hello" == "hello"` sabit havuzu yüzünden `true` olabilir. String içeriği her zaman `equals` ile karşılaştırılır.
- `charAt(0)` ilk karakter, `charAt(s.length()-1)` son karakterdir.
- `substring(2)` 2. indisten sona kadar; `substring(4, 8)` 4..7 arası (8 dahil değil).
- `trim()` baştaki ve sondaki boşlukları siler.
- `Math` metotları statiktir: `Math.random()`, `Math.abs`, `Math.max`, `Math.ceil`, `Math.floor`, `Math.round`, `Math.sqrt`.
- `Random` metotları statik değildir. Önce nesne üretilir: `Random r = new Random(); r.nextInt(5);` -> `0..4`.
- `Scanner sc = new Scanner(System.in);` (`import java.util.Scanner;`). İş bitince `sc.close()`.
- Java'da parametre aktarımı her zaman değerle aktarımdır. Nesnede referansın kopyası gider.
- Dizi metoda gönderilince metot içindeki değişiklik çağıran tarafa yansır.
- `for (int a : dizi)` dizinin her elemanını sırayla `a`'ya kopyalar. `a`'yı değiştirmek diziyi değiştirmez.
- Her sınıf `Object` sınıfından `toString()` alır. Ezilmezse çıktı `paket.Sinif@hashkod` olur.

Detaylı Açıklamalar

1. Paketler ve `import`

- Paketler programı mantıksal modüllere ayırır. Hiyerarşik yapıdadır.
- `import paket1.*;` yalnız `paket1` içindeki sınıfları getirir. `paket1.paket2` içindekiler için ayrıca `import paket1.paket2.*;` yazılır.
- Yalnız alt paket gerekiyorsa üst paketi import etmek gerekmez.

2. Bilgi gizleme neden gerekli?

- Public bir alan her yerden, her zaman değiştirilebilir. Bu, nesnenin tutarlılığını bozar.
- Constructor bir kez çalışır. Set/get metotları kontrollü erişim sağlar.
- Kodunuz kalıtımla veya kullanma ilişkisiyle başka yerlerde kullanılabilir. Alanlar açık olursa projenin geri kalanı bozulabilir.
- Hocanın benzetmesi: Telefonun içindeki devreleri göremeyiz, yalnız bize gereken düğmelere erişiriz.

3. `static` tekrarı

- Statik alan nesneye değil, sınıfa aittir. Bütün nesneler aynı değeri görür.
- Hocanın örneği: kantine giren öğrenci sayısını tutan ortak sayaç. Sayaç sınıra ulaşınca yeni müşteri alınmaz.
- Statik metot sınıf adıyla çağrılır. Statik olmayan metot için önce nesne üretilir.

4. Sarmalayıcı sınıflar ve swap denemesi

- Sarmalayıcı sınıf, tek bir ilkel değeri içinde tutan (kutulayan, box) bir sınıftır. Faydalı metotlar sağlar. Koleksiyonlar ve serileştirme gibi ilkel tipin kullanılamadığı yerlerde ge-

rekir.

- Derste `Integer` ile swap denendi ve **çalışmadı**:

```
static void swap(Integer x, Integer y) {
    Integer t = x;
    x = y;
    y = t;       // yalnız yerel referanslar yer değiştirir
}
// main: Integer x = 3, y = 6; swap(x, y);  -> hâlâ 3 6
```

- Çözüm olarak hoca kendi sarmalayıcı sınıfını yazdı. Swap, set/get ile nesnenin **içeriğini** değiştirdi:

```
class MyClass {
    private int a;
    public MyClass(int a) { this.a = a; }
    public int getA() { return a; }
    public void setA(int a) { this.a = a; }
}

static void swap(MyClass x, MyClass y) {
    int temp = x.getA();
    x.setA(y.getA());
    y.setA(temp);
}
// main: a = new MyClass(4), b = new MyClass(8); swap(a, b);  -> 8 4
```

- Neden çalışıyor? `x` ve `y` yine referansın kopyasıdır. Ama kopya da aynı nesneyi gösterir. Nesnenin içi değişince çağıran taraf bu değişikliği görür.
- Hocanın şeması: `main` içindeki `x` ve `y` referansları 100 ve 110 adreslerini tutsun. `swap` çağrılınca bu adresler yeni yerel değişkenlere kopyalanır. Yerel değişkenlerin yer değiştirmesi `main`'i etkilemez.
- Aynı swap `String` için de çalışmaz. Hoca `String`'i tutan bir `StrClass` yazıp swap'i bu sınıf üzerinden yaptı. Çıktı `merhaba dünya -> dünya merhaba` oldu.
- Global (sınıf düzeyi) değişkenlerle yapılan swap her yerde etkilidir. Ama bu gerçek bir parametre aktarımı değildir.

5. `Integer` metotları

```
Integer t = Integer.parseInt("BDDSF");   // NumberFormatException: For input string: "BDDSF"
Integer sarma1 = Integer.valueOf(1);
Integer sarma2 = Integer.valueOf(7);
System.out.println(sarma1.compareTo(sarma2));   // -1
```

6. `String` sınıfı

- `compareTo` iki metni karakter karakter karşılaştırır. İlk farklı karakterde `karakterA - karakterB` farkını döndürür.
- String sabit havuzu:

```
String aa = "hello";
String bb = "hello";
System.out.println(aa == bb);          // true: ikisi de havuzdaki aynı nesneyi gösterir
System.out.println(aa.compareTo(bb));  // 0
String cc = new String("hello");
System.out.println(aa == cc);          // false: new yeni nesne üretir
System.out.println(aa.equals(cc));     // true: içerik aynı
```

- Karakter dizisinden String: `char[] ch = {'J','a','v','a'}; String s = new String(ch);`. Tersi: `s.toCharArray()`.
- Metot örnekleri:
  - `s.concat("x")` yeni bir String döndürür. `s` değişmez (String değiştirilemez, immutable).

  - `s.contains("abc")` alt metin var mı diye bakar, `boolean` döndürür.
  - `s.endsWith("ship")` sonek kontrolü yapar. Hoca doğal dil işlemede (NLP) sonek ayıklamaya örnek verdi: `relationship -> relation`.
  - `s.indexOf(...)` dört aşırı yüklü biçimi vardır (karakter, metin, başlangıç indisli sürümler).
  - `s.split(regex)` metni düzenli ifadeye göre böler.
  - `"unhappy".substring(2) -> "happy"`. `"hamburger".substring(4, 8) -> "urge"`.
  - `trim()`: Arama kutusuna yalnız boşluk girilirse veritabanı sorgusunda sorun çıkabilir. Önce `trim` yapılır.
- Örnek: `"İstanbul".contains("Yıldız") -> false`.
- Kaynak önerileri: W3Schools ve JavaTpoint String sayfaları.

7. `Math` ve `Random`

- `Math.random()` parametre almaz. `[0.0, 1.0)` aralığında `double` döndürür. Hoca parametreli sürümünü aradı; böyle bir sürüm yoktur.
- `Math.ceil` yukarı, `Math.floor` aşağı yuvarlar. `Math.round` en yakın tam sayıya yuvarlar.
- Sözde rastgele sayı (pseudo-random): Üreteç bir kök (seed) değerinden başlar. Her sayı öncekine bağlıdır. Aynı değere tekrar gelinince dizi tekrarlar. Periyot ne kadar uzunsa üreteç o kadar iyidir.
- `Random` sınıfı `java.util` paketindedir:

```
import java.util.Random;
Random r = new Random();
int a = r.nextInt();     // bütün int aralığı
int b = r.nextInt(5);    // 0..4
double d = r.nextDouble();
boolean e = r.nextBoolean();
```

- Bit işlemleri C ile aynıdır: `&`, `|`, `^`, `~`, `<<`, `>>`. Slayttaki örnek: `n`'in sağdan 4. biti `(n & 8) / 8` veya `(n >> 3) & 1` ile alınır.

8. Giriş/çıkış ve `Scanner` hatası

- `System.out` bir `PrintStream` nesnesidir. `System` sınıfının statik alanıdır. `System.in` giriş, `System.err` hata akışıdır.
- `print` ile `println` arasındaki tek fark satır sonudur.
- `Scanner` hatası:

```
Scanner in = new Scanner(System.in);
System.out.print("How old are you? ");
int age = in.nextInt();          // "32" okunur, Enter tamponda kalır
System.out.print("What is your name? ");
String name = in.nextLine();     // tampondaki boş satırı okur, kullanıcı yazamaz
```

- Çözüm: `nextInt()` sonrasında bir kez boş `in.nextLine();` çağırıp satır sonunu tüketmek. C'deki çözüm `scanf(" %c", ...)` biçimindeki boşluktu.
- Bir sınıfa kendi `main` metodu eklenebilir. Böylece sınıf kendini test eder (slayttaki `Car` örneği).

9. Parametre aktarımı ve diziler

- İlkel tipler değerle aktarılır. Metottaki değişiklik çağırana yansımaz.
- Nesnelerde referansın değeri aktarılır ("call by value of references"). Hoca buna "yalancı call by reference" dedi.
  - Metot nesnenin içini değiştirirse değişiklik kalıcıdır.
  - Metot parametreye yeni nesne atarsa veya iki parametreyi yer değiştirirse çağıran etkilenmez.
- Dizi gönderilince içeriği değişebilir. Slayttaki örnekte ilk ve son eleman yer değiştirdi: `{1,2,3,4,5} -> {5,2,3,4,1}`.
- Karar ve döngü yapıları (`if`, `for`, `while`, `do-while`, `switch`) C ile aynıdır. `do-while` sonunda noktalı virgül gerekir.

- `for-each`:

```
int[] dizi = {1, 2, 3, 4, 5};
int toplam = 0;
for (int a : dizi)
    toplam += a;          // 15
for (Kitap k : kutuphane.kitaplar)
    k.kitapTanit();       // nesne dizilerinde de çalışır
```

10. `toString()` metodunu ezme

- `System.out.println(k)` çağrısı aslında `k.toString()` çağırır.
- `toString()` her sınıfa `Object` sınıfından gelir. Varsayılan sürüm paket adı, sınıf adı ve hash kodunu yazar.
- Hoca `Kitap` ve `Yazar` sınıflarında `toString`'i ezdi:

```
@Override
public String toString() {
    String s = "ID:" + this.id + " AD:" + this.ad;
    if (this.yazar == null)
        s += " Yazar bulunamadi";
    else
        s += " " + this.yazar;    // yazarın toString'i çağrılır
    return s;
}
```

- Bu, kalıtım konusunda ayrıntılı işlenecek. Override, üst sınıftaki tanımın yerine kendi tanımımızı koymaktır.

Ders sonrası notlar

- **Java'da parametre aktarımı:** Slayt anlatımında "Java'da ilkellerde call by value yok", "wrapper class call by reference özelliğine sahip" ve "C'de call by value yoktu" cümleleri geçti. Üçü de yanlış söylenmiştir. Doğrusu:
  - Java'da parametre aktarımı **her zaman** değerle aktarımdır (call by value). İlkellerde değer, nesnelerde referansın değeri kopyalanır.
  - Sarmalayıcı sınıflar call by reference sağlamaz. Derste `Integer` ile swap denemesi bu yüzden çalışmadı (`3 6 -> 3 6`).
  - C'de de varsayılan aktarım değerle aktarımdır. Referansla aktarım pointer ile taklit edilir.
- **Sarmalayıcılar değiştirilemez (immutable):** `Integer` nesnesinin içindeki değeri değiştiren bir set metodu yoktur. `x.intValue() = ...` yazılamaz. `intValue()` bir get metodudur. Bu yüzden `Integer` ile swap mümkün değildir.
- **`static` tekrarı:** Derste "Bir metot static ise bir defaya mahsus initialization yapılıyor" dendi. Bu cümle statik **alan** için geçerlidir. Statik alan sınıf yüklenirken bir kez ilk değer alır.
- **`String` `compareTo` sonucu:** Derste sonucun "0, +1 veya -1" olduğu söylendi. String'de sonuç herhangi bir tam sayı olabilir: ilk farklı karakterlerin farkı ya da uzunluk farkı. Önemli olan işarettir. Örnek: `"a".compareTo("d") -> -3`.
- **`import` kodu büyütmez:** Hoca `import`'un `#include` gibi kodu programa dahil ettiğini söyledi. Doğrusu: `import` yalnız sınıf adlarının kısa yazılmasını sağlar. Java ile denendi: Üç paketi yıldızla import eden ve hiç import içermeyen aynı sınıfın `.class` dosyaları bayt bayt aynı çıktı.
- **`compareTo` sonucunun 0 çıkması:** Hoca bunu havuza bağladı (00:54:20). Sıfır, içerikler aynı olduğu için çıkar. Java ile denendi: `"hello".compareTo(new String("hello"))` da `0` verdi.
- **`trim`:** Hoca "boşlukları yok eder" dedi. `trim` yalnız baştaki ve sondaki boşlukları siler. Java ile denendi: `"  a b  ".trim()` sonucu `"a b"`.
- **`Math.random`:** Hoca slaytı okurken aşırı yüklü sürümleri olduğunu söyledi. Tek sürümdür, parametre almaz.

- **`==` ile String:** `aa == bb` sonucunun `true` olması havuz yüzündendir. `new String("hello")` ile üretilen metin için `==` sonucu `false` olur. Metin karşılaştırmasında her zaman `equals` kullanın.
- **`Integer` önbelleği (derste açıkça anlatılmadı, Java ile denendi):** `Integer` için de benzer bir tuzak vardır. `-128..127` arasındaki değerler önbellekten gelir. `Integer a=127, b=127; a==b` -> `true`, ama `128` için `false`. Sarmalayıcıları `equals` ile karşılaştırın.
- **`Math.round(-2.5)` (derste geçmedi, Java ile denendi):** Java'da sonuç `-2`'dir (`Math.floor(x + 0.5)` mantığı). "5 ve üstü yukarı yuvarlanır" kuralı negatif sayılarda farklı görünür.
- **Hocanın UML sıralama diyagramı düzeltmesi:** Önceki derste "response" denen ok aslında request (mesaj gönderme) okudur. Cevap oku reply/response olarak adlandırılır.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.